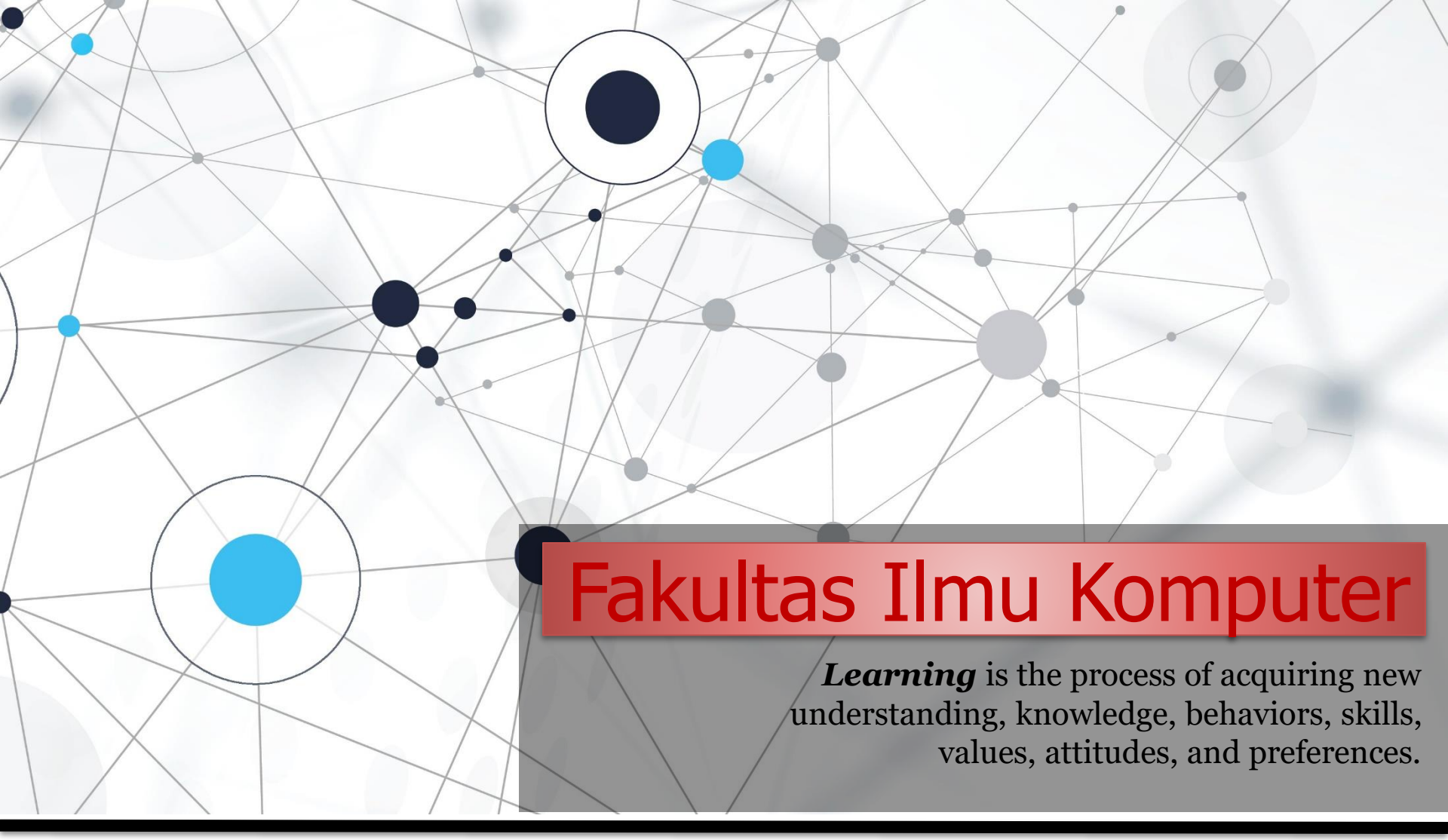
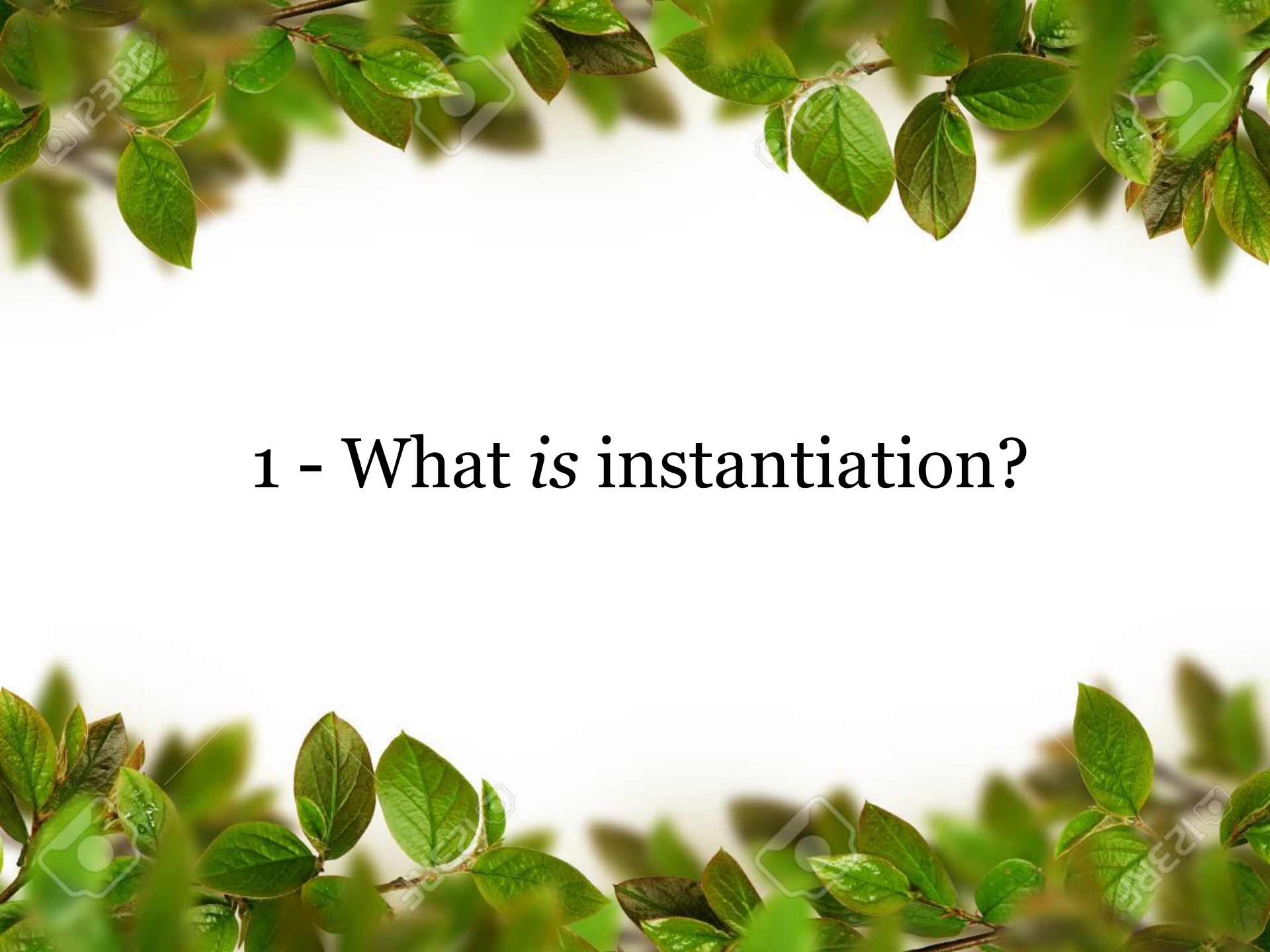


Instantiate an Object *in* Python

Welcome to *the* Object Oriented Programming *course*



1 - What *is* instantiation?

DevOps Programming Comparison

	ADVANTAGES	DISADVANTAGES
Go (Golang)	+ Fast execution speed + Minimal runtime errors due to static typing + Concurrent + Cross-platform + Built-in garbage collection + Single executable	– Less flexible due to static typing – Limited developer familiarity
Python	+ Modular + Flexibility due to dynamic typing + Object-oriented + Extensive third-party libraries and modules	– Slow execution speed – Runtime errors due to dynamic typing – Lower memory efficiency – Potentially complicated runtime environment setup
Scala	+ Similar syntax to Java + Support for highly scalable/concurrent systems + Immutable objects + First-class functions	– Runs in the JVM, which can cause restraints, depending on runtime environment – Limited developer familiarity – Smaller community
Ruby	+ Large and active community + Extensible with many third-party modules + Built-in garbage collection	– Lower performance than compiled languages, notably in boot and execution speed – Documentation less widely available – Tight Active Record coupling
C&C++	+ Fast execution speed + Extensive developer base and documentation + Extensive feature set, depending on language choice	– No garbage collection – Long build times – Different compiler requirements per OS – ABI incompatibility due to compiler requirements

What *is* instantiation in OOP?

- In object-oriented programming (OOP), an **object is essentially an instance of a class**.

- In OOP languages, a class is like a **blueprint/template/design** where variables and methods are defined, and each time a **new instance of the class (*instantiation*) is created**, an **object gets created**, hence the term object-oriented.

- **All objects of a class have** a particular set of associated **variables or properties**, accessories or means to access those variables, and **functions or methods**.

- For example:
  a class may represent a new employee. In this instance, the properties of this class may include job title, role, responsibilities, wages, etc.

What *is* an Object?

Objek (**object**)	Entitas run time yang digunakan untuk menyediakan fungsionalitas pada kelas Python.
Atribut atau property (**variable**)	Didefinisikan di dalam kelas diakses hanya dengan menggunakan objek dari kelas tersebut.
Function **(Method)**	Fungsi di class yang ditentukan pengguna (*user-defined functions*) diakses dengan menggunakan objek.
Konstruktor kelas (**constructor**)	Konstruktor kelas (*constructor*) secara otomatis dipanggil ketika sebuah objek dari kelas tersebut dibuat.
Class object	Setelah kita mendefinisikan atau membuat kelas dengan atribut dan metode, objek kelas baru dibuat dengan nama yang sama dengan kelas tersebut. Objek kelas ini mengizinkan kita untuk mengakses atribut yang berbeda serta menginstansiasi objek baru dari kelas tersebut.

An object consists of:

1) **State** - Attributes or Properties of an object.
2) **Behavior** - Methods of an object.
3) **Identity** - Unique Name to an object and communication between two or more object

What *is* instantiation in OOP?

Sebuah instance dari sebuah objek (object) dapat dideklarasikan dengan memberinya nama unik yang dapat digunakan dalam sebuah program.
Proses ini dikenal sebagai **instansiasi (instantiation)**.
Sebuah kelas juga dapat diinstansiasi untuk membuat sebuah objek, contoh konkret dari kelas tersebut. Objek adalah file yang berisi class yang dapat dieksekusi yang dapat dijalankan di komputer.

An instance of an object can be declared by giving it a unique name that can be used in a program. This process is known as instantiation. A class can also be instantiated to create an object, a concrete instance of the class. The object is **an executable file** that can run on a computer.

What is instantiation in Java?

What is instantiation in C++?

What is instantiation in C#?

What is instantiation in JavaScript?

What is instantiation in Python?

1 - How *to* instantiate an object in Python?

Python's Instantiation Process

- Python classes have a special method called .__new__(), which is responsible for creating and returning a new empty object.
- Then another special method, .__init__(), takes the resulting object, along with the class constructor's arguments.
- The .__init__() method takes the new object as its first argument, self. Then it sets any required instance attribute to a valid state using the arguments that the class constructor passed to it.

Python's instantiation process **starts with a call to the class constructor**, which triggers the **instance creator**, .__new__(), to create a new empty object. The process continues with the **instance initializer**, .__init__(), which takes the **constructor's arguments to initialize** the newly created object.

Python's Instantiation Process

```python
# create class
class Teacher:
    # instance creator (new empty object)
    def __new__(cls, *args, **kwargs):
        print("1. Create a new instance of class Teacher.")
        return super().__new__(cls)

    # constructor (instance initializer)
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print("2. Initialize the new instance of class Teacher.")
        print('My name is {0} ({1} years old).'.format(self.name, self.age));

    # string representator
    def __repr__(self) -> str:
        return f"{type(self).__name__}(name={self.name}, age={self.age})"

    # destructor (object is deleted or destroyed)
    def __del__(self):
        print('3. Object destroyed.')

# create object using constructor
t1 = Teacher('Semmy Taju', 30)
t2 = Teacher('Eugenie Lamansiang', 17)

# delete object
del t1
```

When the class constructor is called, which triggers

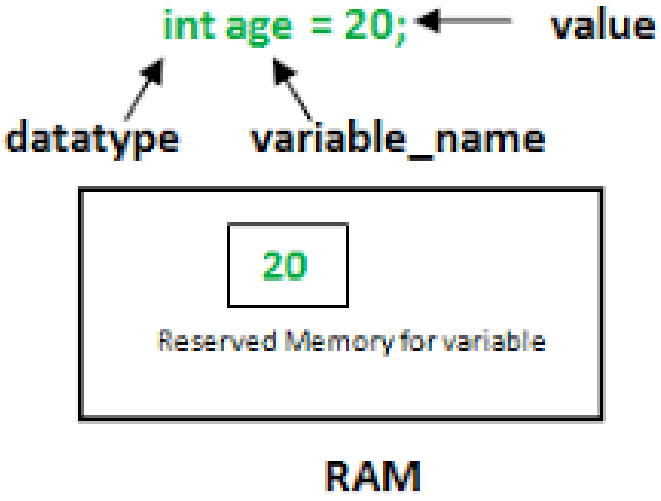
1) The **instance creator**, .__new__(), to create a new empty object.

2) The **instance initializer**, .__init__(), which takes the constructor's arguments to initialize the newly created object.

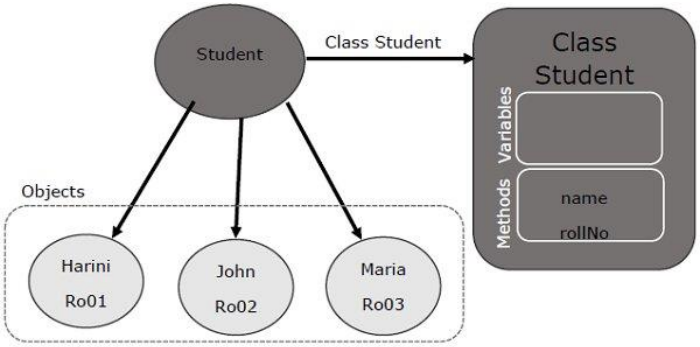
3) The .__repr__() special method, which provides a proper string **representation** for the class.

Special method called *__new__()*, which is responsible for creating and returning a new empty object. Then another special method, *__init__()*, takes the resulting object, along with the class constructor's arguments.

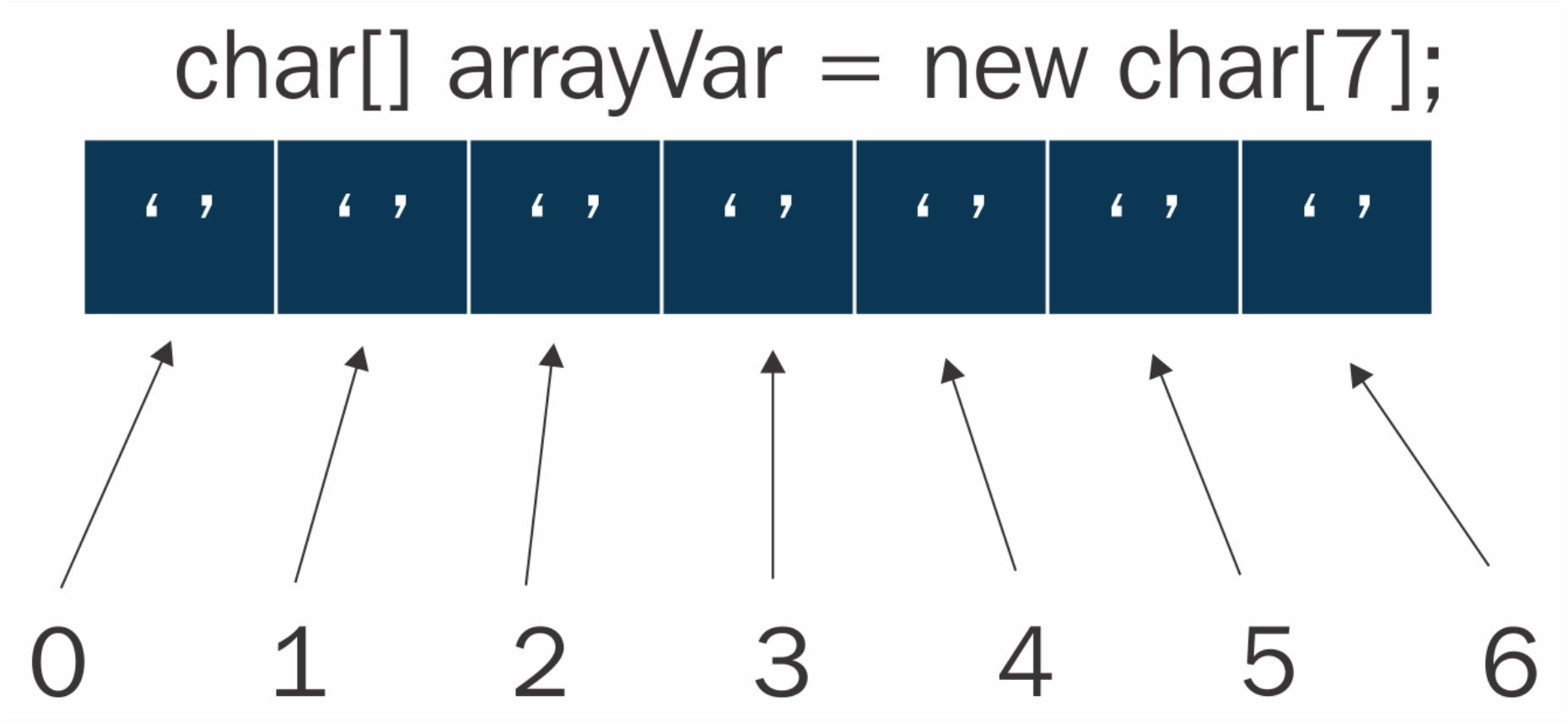
"instantiated" *vs* "initialized"



(1) Variables are Initialized:
All variables are always given an initial value at the point the variable is declared. Thus all variables are initialized. For example, **int's initialize to zero by default**.



(2) Objects are Instantiated:
An object is an instance of some Class. **The act of creating an instance of a Class is called instantiation**. For example, a baby is an instance of a Human.



Baris kode diatas adalah deklarasi dan inisialisasi array karakter (**char**) dengan nama **arrayVar** yang memiliki panjang 7. Array ini akan menyimpan 7 karakter.

Declare, Initialize, & Instantiate Come Together

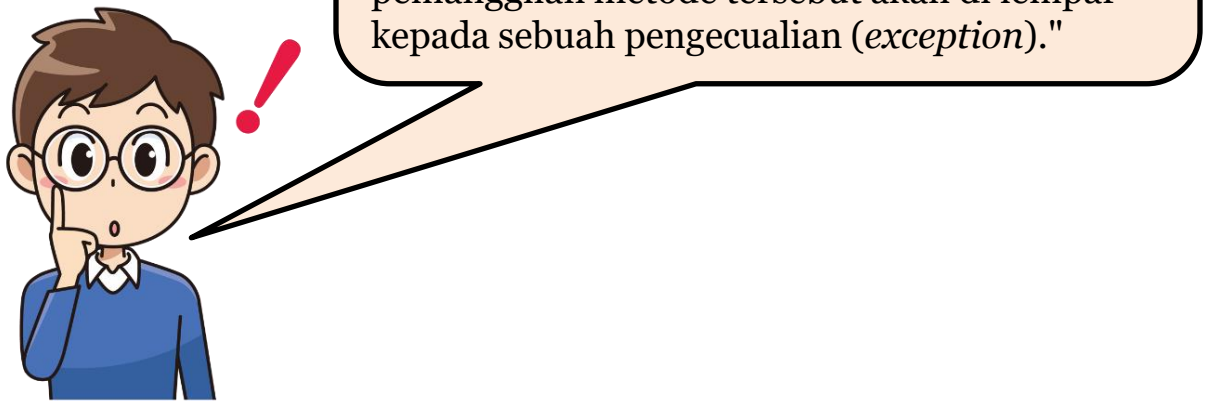
Perhatikan Java Class berikut ini:

MyClass myClassReference = new MyClass();

1) A reference variable is declared ⇒ **MyClass myClassReference**
2) An object is **instantiated** (...from/of a given class, implied) ⇒ **new MyClass()**
3) The object is assigned/initialized to the variable ⇒ Symbol **"=" is used**.

Example:

MyClass *myClassReference*;
myClassReference.DoSomething();



Teacher Questions

Perhatikan Java Class berikut ini:

MyClass myClassReference = new MyClass();

Example:

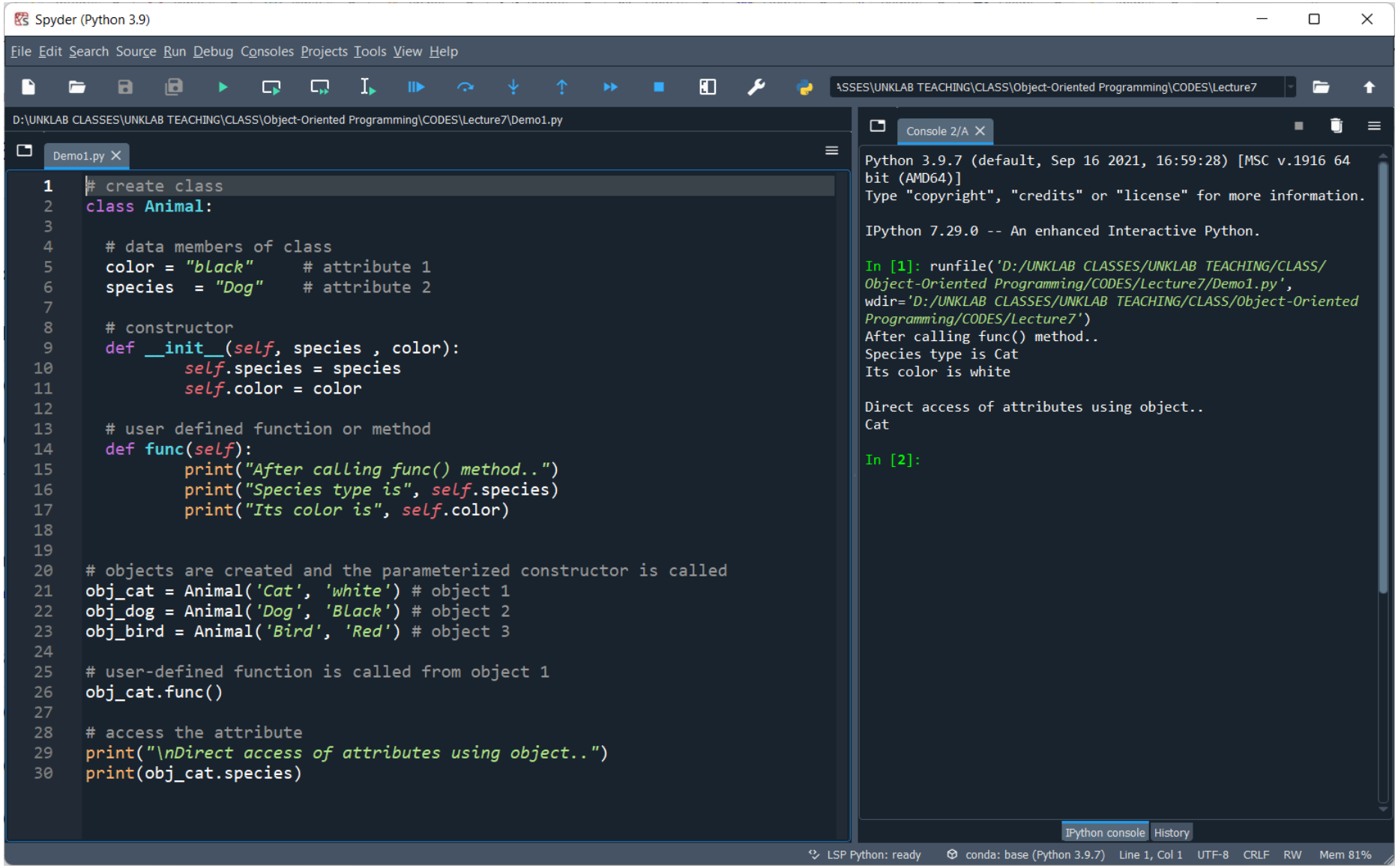
MyClass *myClassReference*;
***myClassReference*.DoSomething();**

Bagaimana jika seperti ini???
Apakah bisa??? Bagaimana cara menyebutkannya?

MyClass myClassReference = null;

Exercise *for* Students

Exercise *for* Students



Modifikasi source code pada file Demo1.py:

1. Dalam **class Animal**, tambahkan *class variable* (***name, age, weight, location, gender, health_status***) yang merupakan atribut mewakili karakteristik atau informasi tentang class Animal dengan type String yang sudah di inisialisasi (***default initial value***). Perbaiki dan bedakan antara species (*jenis atau spesies*) dan name (nama hewan) dari class Animal.
   1. **Mamalia**: Contohnya manusia, gajah, singa, dan lumba-lumba.
   2. **Burung**: Seperti burung camar, elang, merpati, dan burung hantu.
   3. **Reptil**: Misalnya ular, kura-kura, buaya, dan kadal.
   4. **Amfibi**: Seperti katak, salamander.
   5. **Ikan**: Contohnya salmon, hiu, paus, dan ikan mas.
   6. **Serangga**: Seperti semut, lebah, capung, dan kecoak.
   7. **Arachnida**: Misalnya laba-laba, kalajengking, dan lipan.
   8. **Invertebrata Laut**: Seperti ubur-ubur, teripang, dan krustasea.
   9. **Hewan Avertebrata**: Misalnya cacing, remis, dan spons.

2. ~~Students diharapkan untuk dapat berdiskusi dan menambahkan atribut-atribut lain yang bervariasi yang dalam mewakili class Animal. Setiap class variable tersebut harus sudah mempunyai *default initial value*.~~

3. Buat/tambah method-method baru (*get_name()* get_age*(), get_weight(), get_location(), get_gender(), get_health()*). Dimana method-method tersebut tidak memiliki parameter tapi dapat mengembalikan nilai dari *variable-variable* di point-1 dengan type String.

4. Sebuah method *show_informations()* digunakan untuk **menampilkan** semua informasi atribut mewakili karakteristik class Animal, harus menggunakan keyword ***self*** dari dalam class dan informasi atribut ditampilkan dari method-method yang telah di buat pada point #3. Method *show_informations()* tidak mengembalikan nilai.

5. Dari class Animal, buat **8 object baru** sebagai berikut (Ular, Salmon, Semut, Elang, Gajah, Lipan, Buaya dan Singa) dalam Bahasa inggris.

6. Tampilkan output dari method *show_informations()* menggunakan **setiap object** dari class Animal.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T